request

DrawTrainCardRequest
- slot: int

ClaimRouteRequest
- routeId: int
- selectedTrainCardIds: List<Integer>

KeepDrawDestRequest
- keepDestCardsIds: List<Integer>
- discardDestCardsIds: List<Integer>

DrawDestCardRequest

<<interface>> ActionRequest
+ buildCommand(game: Game): ActionCommand

<<interface>> GameUI
+ onTrainCardClicked(slot: int): void
+ onClaimRouteClicked(routeId: int, cardIds: List<Integer>): void
+ onDrawDestinationCardsClicked(): void
+ onTicketsSelectionConfirmed(keepIds: List<Integer>, discardIds: List<Integer>): void

setup

PlayerConfiguration
- playerName: String
- playerColour: PlayerColour
+ getPlayerName(): String
+ getPlayerColour(): PlayerColour

GameConfiguration
- playerConfigList: List<PlayerConfiguration>
- map: MapType
+ getPlayerConfigs(): List<PlayerConfiguration>
+ getMapType(): MapType

<<enumeration>> MapType
LONDON

LondonGameBuilder
- newGame: Game
- destTieBreakerOrder: int {readOnly}
- longestRouteTieBreakerOrder: int {readOnly}

<<interface>> GameBuilder
+ buildBoard(): void
+ buildDecks(): void
+ buildPlayers(playerConfigs: List<PlayerConfiguration>): void
+ buildServices(): void
+ dealStartingHands(): void
+ getResult(): Game

GameSetupDirector
+ constructGame(config: GameConfiguration): Game

main

<<interface>> PendingDecisionVisitor
+ visitTicketDecision(decision: TicketDecision): void

GameController
- game: Game
+ onStartGame(config: GameConfiguration): void
+ handleRequest(actionReq: ActionRequest): ActionResult
+ notifyPendingDecision(pendingDecision: endingDecision): void
+ endGame(): void
- processActionResult(actionResult: ActionResult): void
- processEndTurnResult(endTurnResult: EndTurnResult): void

Game
- players: List<Player>
- currentPlayerIndex: int
- board: Board
- trainCardDeck: TrainCardDeck
- destinationCardDeck: DestinationCardDeck
- endGameService: EndGameService
- routeScoringStrategy: RouteScoringStrategy
- turnMediator: TurnMediator
- lastRoundTracker: LastRoundTracker
+ play(request: ActionRequest): ActionResult
+ endTurn(): ActionResult
+ getCurrentPlayer(): Player
+ isGameOver(): boolean
+ resolveGameOver(): List<Player, List<Player>>
+ getTrainCardDeck(): TrainCardDeck
+ getDestinationCardDeck(): DestinationCardDeck
+ getPlayerByIndex(ind: int): Player

deck

<<abstract>> Deck
- inDeckPile: List<Card>
+ draw(): Card
+ shuffle(): void
+ getInDeck(): List<Card>
+ isInDeckEmpty(): boolean

DestinationCardDeck
+ drawThree(): List<DestinationCard>
+ returnToBottom(cards: List<DestinationCard>): void

TrainCardDeck
- discardedPile: List<TrainCard>
- faceUpPile: List<TrainCard>
- IN_DECK_SLOT: int {read_only}
+ canDraw(slot: int): boolean
+ draw(slot: int): TrainCard
- drawFaceUp(slot: int): TrainCard
- drawInDeck(): TrainCard
+ isFaceUpEmpty(): boolean
+ getFaceUpCards(): List<TrainCard>
+ discard(cards: List<TrainCard>): void
+ canDrawFaceUp(slot: int): boolean
+ peekFaceUp(slot: int): TrainCard
- checkAndReplaceLocomotives(): void
- shuffleDiscardIntoDeck(): void
- replaceFaceUp(slot: int): void
- tryRefillFaceUpPile(): void
- tryRefillInDeckPile(): void

card

<<abstract>> Card
- id: int
+ getId(): int

TrainCard
- colour: CardColour {readOnly}
+ getColour(): CardColour
+ isLocomotive(): boolean

<<enumeration>> CardColour
ORANGE
YELLOW
GREEN
BLUE
PINK
BLACK
WILD

DestinationCard
- from: City
- to: City
- points: int
+ getFrom(): City
+ getTo(): City
+ getPoints(): int
+ isCompleted(ownerPlayer: Player, board: Board): boolean

map

Board
- cities: List<City>
- routes: List<Route>
+ getRoute(from: City, to: City): Route
+ getRouteById(routeID: string): Route
+ getRoutesClaimedBy(player: Player): List<Route>
+ isPathConnected(player: Player, from: City, to: City): boolean
+ getAvailableRoutes(): List<Route>

Route
- id: String
- cityA: City
- cityB: City
- length: int
- colour: RouteColour
- isClaimed: boolean
- owner: Player
- claimStrategy: RouteClaimStrategy
- routeGroup: RouteGroup
+ getId(): String
+ getCityA(): City
+ getCityB(): City
+ getLength(): int
+ getColour(): RouteColour
+ isClaimed(): boolean
+ setOwner(player: Player): void
+ getOwner(): Player
+ getClaimStrategy(): RouteClaimStrategy

City
- name: String
- x: int
- y: int
+ getName(): String
+ getX(): int
+ getY(): int

<<enumeration>> RouteColour
ORANGE
YELLOW
GREEN
BLUE
PINK
BLACK
GREY

RouteGroup
- routeInParallel: List<Route>
- playerCount: int {read only}
+ canClaim(Player, Route): boolean

scoring

<<interface>> RouteScoringStrategy
+ score(route: Route): int

EndGameService
- finalScores: Map<Player, Integer>
- endGameScoring: List<EndGameScoringStrategy>
- tieBreakerStrategy: List<TieBreakerStrategy>
+ endGameScoring(players: List<Player>, board: Board): void
+ determineWinner(players: List<Player>, board: Board): Player

<<interface>> EndGameScoringStrategy
+ apply(players: List<Player>, board: Board): void

<<interface>> TieBreakerStrategy
+ getTieBreaker(List<Player>, Board): List<Player>
+ getTieOrder(): int

StandardRouteScoring
- scoreTable: Map<Integer, Integer> {readOnly}

DestinationTicketScoring
- playerDestCompletedCount: Map<Player, Integer>
- tieOrder: int {readOnly}

LongestRouteScoring
- bonusPoints: int = 10 {readOnly}
- bonusPlayer: Set<Player>
- tieOrder: int {readOnly}
- computeLongestPath(player: Player, board: Board): int

character

Player
- name: String
- colour: PlayerColour
- remainingTrainPieces: int
- score: int
- hand: PlayerHand
+ getHand(): PlayerHand
+ getScore(): int
+ getRemainingTrainPieces(): int
+ getName(): String
+ getColour(): PlayerColour
+ addScore(points: int): void
+ hasEnoughTrainPieces(count: int): boolean
+ reduceTrainPieces(numOfPieces: int): void
+ claimRoute(Route, List<TrainCard>, RouteScoringStrategy): void
+ drawTrainCard(TrainCard): void
+ drawDestCard(List<DestinationCard>): void

<<enumeration>> PlayerColour
RED
BLUE
GREEN
YELLOW
BLACK

PlayerHand
- trainCards: Map<Colour, List<TrainCard>>
- destinationCards: List<DestinationCard>
+ getTrainCards(): Map<Colour, List<TrainCard>>
+ getDestinationCards(): List<DestinationCard>
+ getLocomotiveCount(): int
+ getTrainCardCountByColour(colour: Colour): int
+ addTrainCards(cards: List<TrainCard>): void
+ addDestinationCards(cards: List<DestinationCard>): void
+ removeTrainCards(cards: List<TrainCard>): void
+ removeDestinationCards(cards: List<DestinationCard>): void
+ hasEnoughOfAnyColour(needed: int): boolean
+ getTrainCardByIds(ids: List<Integer>): List<TrainCard>
+ getDestinationCardByIds(ids: List<Integer>): List<TrainCard>

strategy

<<interface>> RouteClaimStrategy
+ validate(player: Player, route: Route, spentCards: List<TrainCard>): boolean

GrayRouteClaimStrategy

StandardRouteClaimStrategy

turnManager

TurnTracker
- triggered: boolean
- turnsRemaining: int
- triggeringPlayer: Player
- totalPlayers: int
+ endTurn(currentPlayer: Player): EndTurnOutcome
+ checkAndTrigger(currentPlayer: Player): boolean
+ decrementRemainingTurns(): void
+ isTriggered(): boolean
+ isOver(): boolean
+ getTriggeringPlayer(): Player

TurnMediator
- remainingActionCost: int
- pendingDecision: PendingDecision
- maxActionCost: int = 2 {readOnly}
+ isTurnComplete(): boolean
+ processAction(action: ActionCommand, currentPlayer: Player): ActionResult
- clearPendingDecision(): void
- reset(): void

command

<<enumeration>> ActionCost
FREE
FULL
HALF
- cost: int
+ getCost(): int

<<interface>> ActionCommand
+ execute(player: Player): ActionResult
+ getCost(): ActionCost

<<enumeration>> ActionStatus
SUCCESS
FAILED
PENDING
PENDING_FAILED

ActionResult
- status: ActionStatus
- message: String
- endTurnResult: EndTurnResult
- pendingDecision: PendingDecision
+ isSuccess(): boolean
+ getEndTurnResult(): EndTurnResult
+ static success(message: String): ActionResult
+ static fail(message: String): ActionResult
+ static pending(message: String, decision: PendingDecision): ActionResult
+ setEndTurnResult(EndTurnResult): void
+ getPendingDecision(): PendingDecision
+ hasPendingDecision(): boolean

EndTurnResult
- outcome: EndTurnOutcome
- nextPlayer: Player
+ static nextTurn(nextPlayer: Player): EndTurnResult
+ static lastRoundTriggered(nextPlayer: Player): EndTurnResult
+ static gameOver(): EndTurnResult
+ getOutcome(): EndTurnOutcome
+ static lastRoundContinues(nextPlayer: Player): EndTurnResult

<<enumeration>> EndTurnOutcome
NEXT_PLAYER_TURN
LAST_ROUND_TRIGGERED
LAST_ROUND_CONTINUES
GAME_OVER

DrawTrainCardCommand
- slot: int
- trainCardDeck: TrainCardDeck

ClaimRouteCommand
- route: Route
- selectedTrainCards: List<TrainCard>
- trainCardDeck: TrainCardDeck
- scoring: RouteScoringStrategy

DrawDestCommand
- ticketDeck: DestinationCardDeck

KeepDestCommand
- keepDestCards: List<DestinationCard>
- discardDestCards: List<DestinationCard>
- ticketDeck: DestinationCardDeck

<<interface>> PendingDecision
+ accept(visitor: PendingDecisionVisitor): void

TicketDecision
- drawnCards: List<DestinationCard>
- minKeep: int
+ getDrawnCards(): List<DestinationCard>
+ getMinKeep(): int
+ getCardsbyIds(ids: List<Integer>): List<DestinationCard>